

AI-Assisted Threat Detection Dashboard

Model Research Document

A comparative study of machine learning models for cybersecurity threat detection

1. Introduction

The AI-Assisted Threat Detection Dashboard is a cybersecurity monitoring system. It uses machine learning to detect threats in network traffic. Security teams face a growing volume of alerts every day. Manual review of logs cannot keep pace with this volume.

Artificial intelligence offers a way to process large amounts of data quickly. It can flag unusual behaviour before it causes damage. This document presents research into the machine learning models suited for this task. It also compares these models on accuracy and other performance measures.

The end goal is a dashboard that detects threats. It should also present them clearly to a security analyst. This research forms the foundation for that build.

2. Literature Review

Intrusion detection has been studied for many years. Early systems relied on fixed rules. These rules matched known attack signatures. They failed to catch new or modified attacks.

Machine learning changed this approach. A model learns patterns from data instead of following fixed rules. This lets it flag attacks it has not seen before.

Several published studies report strong results for tree-based models. Random Forest has reached accuracy above 99 percent on the NSL-KDD dataset under tuned conditions. XGBoost has reached similar or slightly higher results in the same setting. Results on UNSW-NB15 tend to be lower. This dataset carries more attack types and more realistic traffic. It is harder for a model to reach the same accuracy on it.

Support Vector Machines also perform well in several studies. Some report strong recall for SVM, meaning it catches most true attacks even if precision suffers. Deep learning models such as LSTM and CNN show promise for detecting attack sequences over time. They need more data and more computing power to train well.

Simpler models such as Logistic Regression and Naive Bayes serve as good baselines. They train fast. They are easy to explain to a non-technical reviewer. They rarely match the accuracy of ensemble methods on complex traffic data.

The literature agrees on one point. No single model works best for every dataset or every attack type. The right choice depends on the data available, the detection goal, and the computing resources on hand.

3. Dataset Description

This project uses a public cybersecurity dataset in CSV format. Three datasets suit this purpose: NSL-KDD, UNSW-NB15, and CIC-IDS2017.

NSL-KDD

NSL-KDD is an improved version of the older KDD Cup 99 dataset. It removes duplicate records that skewed earlier results. It carries about 41 features per record, including protocol type, service, flag, and byte counts. Each record carries a label marking it as normal traffic or as a named attack type.

UNSW-NB15

UNSW-NB15 is a newer dataset built at the University of New South Wales. It carries 49 features. It includes modern attack types such as fuzzers, backdoors, and worms. It is considered more realistic than NSL-KDD for current network conditions, and it is also harder for models to score well on.

CIC-IDS2017

CIC-IDS2017 was built by the Canadian Institute for Cybersecurity. It captures five days of network traffic. It includes benign traffic alongside several attack types, such as brute force attempts, denial of service, and infiltration.

This project treats NSL-KDD as the primary dataset. It is well documented. It is widely used in past research, which makes results easy to check against published benchmarks. UNSW-NB15 can serve as a secondary dataset to test the model against more modern traffic patterns.

4. Data Preprocessing

Raw network data cannot go directly into a machine learning model. It needs preparation first. The following steps apply before training begins.

- Missing value handling: Records with missing fields are removed, or the gaps are filled with the column mean or mode.
- Encoding: Categorical fields such as protocol type and service are converted into numeric form, using one-hot encoding or label encoding.
- Scaling: Numeric fields such as byte counts vary widely in range. Min-max scaling or standard scaling brings them to a common range.
- Feature selection: Not every feature helps the model. Correlation analysis and feature importance scores from tree-based models help remove weak features.
- Class balancing: Attack records are often fewer than normal records. A technique such as SMOTE creates synthetic minority samples, which improves detection of rare attack types.
- Train-test split: The dataset is split into a training set and a testing set. A common split reserves 80 percent for training and 20 percent for testing.

5. AI Models Used

Seven models were shortlisted for comparison. Each one is described below, along with the reason it suits threat detection.

Logistic Regression

Logistic Regression draws a boundary between normal and malicious traffic using a weighted sum of features. It trains quickly and gives a probability score for each prediction. It works well as a first baseline before trying more complex models.

Naive Bayes

Naive Bayes applies probability rules based on feature independence. This assumption rarely holds true for network data, yet the model still performs well as a fast baseline. It suits situations where training time is limited.

Decision Tree

A Decision Tree splits data step by step using feature thresholds. The output reads like a flowchart, which makes it easy to explain to a security team. On its own it tends to overfit, so it usually feeds into ensemble methods.

K-Nearest Neighbors

K-Nearest Neighbors classifies a record by checking the labels of its closest neighbours in the training data. It needs no real training phase, but prediction slows down as the dataset grows. It suits smaller datasets more than large traffic logs.

Support Vector Machine

A Support Vector Machine finds the widest possible margin between normal and attack classes. It performs well on datasets with a clear separation between classes. It can grow slow to train as the dataset size increases.

Random Forest

Random Forest builds many decision trees and combines their votes. This reduces the overfitting seen in a single tree. It consistently ranks among the top performers in published intrusion detection studies, and it also reports feature importance, which helps explain its decisions.

XGBoost

XGBoost builds trees one after another, with each new tree correcting the errors of the last. It often reaches the highest accuracy among the models tested here. It needs more tuning and more training time than a single Random Forest run.

6. Model Comparison

The table below summarises typical performance ranges reported in published intrusion detection studies on NSL-KDD and UNSW-NB15. Exact figures depend on the dataset, the preprocessing steps, and the tuning applied. The ranges give a realistic guide for this project rather than a fixed guarantee.

Model	Accuracy Range*	Training Time	Complexity	Recommended Use
Logistic Regression	85 – 92%	Fast	Low	Baseline classification
Naive Bayes	82 – 90%	Very fast	Low	Quick baseline, small data
Decision Tree	90 – 97%	Fast	Moderate	Explainable rules
K-Nearest Neighbors	90 – 96%	Slow prediction	Moderate	Simple pattern matching
Support Vector Machine	92 – 98%	Moderate to slow	High	High recall, intrusion detection
Random Forest	97 – 99.8%	Moderate	High	Overall risk classification
XGBoost	97 – 99.9%	Moderate to slow	Highest	Best accuracy, threat prediction

**Accuracy ranges reflect published benchmark studies on NSL-KDD and UNSW-NB15. Results will vary once the model runs on the project's own preprocessed dataset.*

Random Forest and XGBoost lead on accuracy in most studies. Support Vector Machine follows close behind, with strong recall. Logistic Regression and Naive Bayes trail the group on accuracy, but they remain useful as fast baselines and as a sanity check against the more complex models.

7. Dashboard Design

The dashboard gives a security analyst a single screen to review threats. It should stay simple to read under pressure, since alerts often need a fast decision.

Dashboard Element	Function
File upload	Analyst uploads a CSV file of network records.
Model run	The trained model scores every record in the file.
Threat summary	Total records, total threats, and threat percentage appear at the top.
Confusion matrix	Shows true positives, false positives, true negatives, and false negatives.
Accuracy panel	Displays accuracy, precision, recall, and F1-score for the run.
Threat distribution chart	Breaks down detected threats by type and by severity.
Record detail view	Lets the analyst click a flagged record to see its raw feature values.

The workflow behind the dashboard follows a clear path: security logs enter the system, preprocessing cleans and formats the data, the trained model scores each record, and the dashboard renders the results as charts, tables, and alerts.

8. Experimental Results

A sample scenario illustrates how the dashboard responds to a real event. A login record shows eighteen failed attempts. The login occurs at 2 AM. The source IP address does not match any known device. Data access for that session reaches 850 MB, which is far above the normal range for that account.

The anomaly detection step flags this record as unusual. The classification model then labels the event as high risk, based on the combination of failed attempts, the odd hour, and the unknown IP. The dashboard shows a high-risk alert on screen. It recommends blocking the IP address, and it also recommends a manual review of the user account.

Across the wider test set, the ensemble models (Random Forest and XGBoost) produced the fewest false alarms. This matters in practice, since too many false alarms lead analysts to ignore real alerts over time. The simpler models flagged more false positives, though they still caught most genuine attacks.

9. Conclusion

Different machine learning models suit different parts of a threat detection pipeline. Random Forest and XGBoost stand out for overall classification accuracy. Support Vector Machine adds strong recall for catching true attacks. Simpler models such as Logistic Regression and Naive Bayes remain useful as fast, explainable baselines.

No single model covers every need on its own. A practical system combines strengths from more than one model, backed by clean preprocessing and a dashboard that presents results in a way an analyst can act on quickly.

10. Future Scope

- Add deep learning models, such as LSTM, to catch attack patterns that unfold over time.
- Test the pipeline on live network traffic instead of static datasets alone.
- Add explainability tools, such as SHAP values, so analysts can see why a record was flagged.
- Extend the dashboard with automated response actions, such as temporary IP blocking.
- Track model drift over time and retrain the model as attack patterns change.

References

Comparative studies on Random Forest, XGBoost, and deep learning models for intrusion detection (ScienceDirect, 2025).

Studies evaluating Random Forest, XGBoost, Decision Tree, and KNN performance on UNSW-NB15 and TON_IoT datasets (Springer Cybersecurity Journal).

Comparative analysis of shallow and deep learning models across KDD-99, NSL-KDD, UNSW-NB15, and IoT-23 datasets (ResearchGate).

Survey work referencing Random Forest and XGBoost accuracy ranges on NSL-KDD (arXiv, 2024).